

Employee Leave Management System

Project Specification & System Architecture Document

Elevator Pitch

A company wants to digitize its leave approval process.

- Employees should be able to submit leave requests.
- Managers approve or reject requests.
- HR has the final approval.
- Admins manage the entire system.

Sprint 0 (Before Day 1)

*Real companies don't start by opening Visual Studio.
They start with requirements.
Today we're doing Sprint 0.*

Product Requirements

Our users are:

Role	Can	Cannot
Employee	• Login • View profile • Submit leave request • Cancel pending request • View leave history • View remaining leave balance	• Approve requests • Modify approved requests
Manager	• Login • View team leave requests • Approve • Reject • Add comments	• Manage users • Change leave balances
HR	• Final approval • Reject • Manage company holidays • Manage leave types	—
Admin	• Can do everything.	—

Functional Requirements

The system must support:

Authentication

- Register
- Login
- JWT
- Refresh Tokens (stretch goal)

Employee Management

- CRUD Employees
- Departments
- Managers

Leave Requests Workflow

Employee creates request → Manager approves → HR approves → Completed

Leave Types

- Annual, Sick, Unpaid, Remote Work, Maternity, etc.

Leave Balance Rules

Every employee has:

- **Annual Leave:** 21 days
- **Sick Leave:** 15 days
- **Unpaid Leave:** Unlimited

When a leave request is approved, balance decreases automatically.

Company Holidays

HR can create company holidays (e.g. Christmas, New Year, National Holidays).

These should affect leave calculations if we decide to implement working-day calculations later.

Audit Logs

Every important action must be logged.

- **Example:** John approved LeaveRequest #45
- **Logged Fields:** Time, User, IP, Action

Non-functional Requirements

We want:

- ✓ Clean Architecture
- ✓ SOLID Principles
- ✓ Logging
- ✓ Global Exception Handling
- ✓ Validation
- ✓ Swagger
- ✓ Docker (later)
- ✓ Unit Tests (later)

Technology Stack

.NET 9, ASP.NET Core, EF Core, SQL Server, Identity, JWT, AutoMapper, FluentValidation, Serilog, Docker, Swagger.

Architecture

We'll use Clean Architecture, but with a practical mindset. Instead of adding every pattern just because it's popular, we'll ask whether it adds value.

Our solution will likely look like this:

```
EmployeeLeaveManagement.sln
src/
  ├── EmployeeLeaveManagement.API
  ├── EmployeeLeaveManagement.Application
  ├── EmployeeLeaveManagement.Domain
  └── EmployeeLeaveManagement.Infrastructure
tests/
  ├── EmployeeLeaveManagement.UnitTests
  └── EmployeeLeaveManagement.IntegrationTests
```

Git Strategy

We'll work like a professional team. Never commit directly after a huge change.

Example sequence of small, meaningful commits:

- Initial Clean Architecture setup
- Configure EF Core
- Implement Identity
- Add JWT Authentication
- Implement Employee CRUD

Development Rules

These are our team rules:

- Controllers stay thin.
- Business logic belongs in the Application layer.
- Data access belongs in Infrastructure.
- Domain entities don't depend on EF Core or ASP.NET Core.
- Avoid "magic strings" and duplicated logic.
- Every feature should compile and run before moving to the next.